[Sign (1b) | Exponent (5b) | Mantissa (10b)]

STEP 1: EXTRACTION

- sign: use bitwise AND mask to mask out 15th bit (0x8000)
- exponents: use bitwise AND mask to extract bit 14-10 (0x7c00) then shift right by 10 (turns into a normal num)
- fraction: use AND mask (0x03ff) to mask out bit 9-0
- OR the value 0x0400 in fraction part
- this converts fraction into normalized version

STEP 2: ALIGNMENT

- cant add mantisas if exponents are diff
- compare both exponents and find the absolute diff **E**
- shift smaller mantissa right by **E** so both match
- TOY limitation: Implement a software loop to sequentially shift right by 1 bit E times

STEP 3:ADD

- Add or subtract the mantissas using standard integer math.

STEP 4:NORMALIZE

- Check if the result overflowed or underflowed. Shift to restore the normalized format and adjust the exponent
- if bit 11 is active (overflowed) shift the mantissa right and increment exponent by 1
- if underflow shift mantissa left until a 1 lands back at position 10
- decrement exponent

STEP 5:PACKING

- clear bit 10 of resulting mantissa by bitwise AND (0x03ff) this compresses your mantissa back down into a standard 10-bit fraction avoiding overflow (fraction bits 9-0, exponent bits 14-10 so 10 should be free)
- shift resulting exponent left by 10 pos
- Use bitwise OR masks to fuse the pieces back into a single 16-bit word.

PSEUDOCODE: addFloat(A, B)

EXTRACT (mask / shift)

- sign = (x & 0x8000)
- exp = (x & 0x7C00) >> 10
- mantissa =if x != 0: mantissa = (x & 0x03FF) |0x0400

ALIGN (bit shift)

- diff = expBig - expSmall
- while diff > 0:

  manSmall = manSmall >> 1 // TOY-8 hardware loop

  diff = diff - 1

ARITHMETIC (integer ALU)

- if signA == signB:
- mantissa = mantissaBig + mantissaSmall
- else:
- mantissa = mantissaBig - mantissaSmall

NORMALIZE (shift + adjust exp)

- if bit 11 set: mantissa >>= 1; exp += 1
- while bit 10 clear: man <<= 1; exp -= 1

PACK (OR merge)

- mantissa = mantissa & 0x03FF
- result = (sign<<15) | (exp<<10) | mantissa